Final Project: First Check-In

Camila Torres

2025-09-29

R Markdown

Data Source: For your final project, you will be working with real RNA-seq count and meta data from breast cancer patients enrolled in The Cancer Genome Atlas (TCGA) Breast Invasive Carcinoma (TCGA-BRCA) cohort. This data encompasses diverse subtypes, tumor stages, and patient demographics. Some samples represent tumor tissues; others matched normal control samples from the same patient. TCGA is a landmark cancer genomics program jointly funded by the National Cancer Institute (NCI) and the National Human Genome Research Institute (NHGRI). TCGA generated comprehensive molecular profiles for over 11,000 patients across 33 cancer types, including bulk RNA-seq DNA methylation, copy number variation, somatic mutations, and clinical annotation.

This data source includes the following meta data:

• Patient Demographics • Tumor Characteristics • Treatment Information • Survival Data • Sample Details

First Check-In

1. Data Preparation: Download the Sample RNA-seq Count Matrix and associated Metadata.

a. Ensure that you have both the count matrix and the metadata file available for your analysis.

```
# Load data
count_data <- read.csv("~/Desktop/Intro to DS/final project/counts.csv", row.names = 1)
metadata <- read.csv("~/Desktop/Intro to DS/final project/meta_data.csv", row.names = 1)
```

2. Gene Selection and Summary Statistics:

a. Select One Gene: choose a gene from the dataset that interests you.

```
# Selecting two genes for gene count
gene1_counts <- as.numeric(count_data["ENSG00000000003.15", ])
gene2_counts <- as.numeric(count_data["ENSG00000000005.6", ])
```

b. Generate Summary Statistics: Using the count data from the selected gene, compute and report summary statistics, such as mean, median, standard deviation, minimum, and maximum.

```
# Getting counts for my gene
gene1_counts <- as.numeric(count_data["ENSG00000000003.15", ])

# Summary stats with label
cat("Summary:",summary(gene1_counts), "\n")
```

```
## Summary: 69 1561.5 2700 3208.132 4144.5 40780
```

```
# Other stats with label
cat("Mean:", mean(gene1_counts), "\n")
```

```
## Mean: 3208.132
```

```
cat("Median:", median(gene1_counts), "\n")
```

```
## Median: 2700
```

```
cat("Standard Deviation:", sd(gene1_counts), "\n")
```

```
## Standard Deviation: 2510.336
```

```
cat("Minimum:", min(gene1_counts), "\n")
```

```
## Minimum: 69
```

```
cat("Maximum:", max(gene1_counts), "\n")
```

```
## Maximum: 40780
```

3. Visualization:

a. Create a Histogram: Use ggplot2 to generate a histogram of the count data for the selected gene. This visualization should effectively display the distribution of the counts.

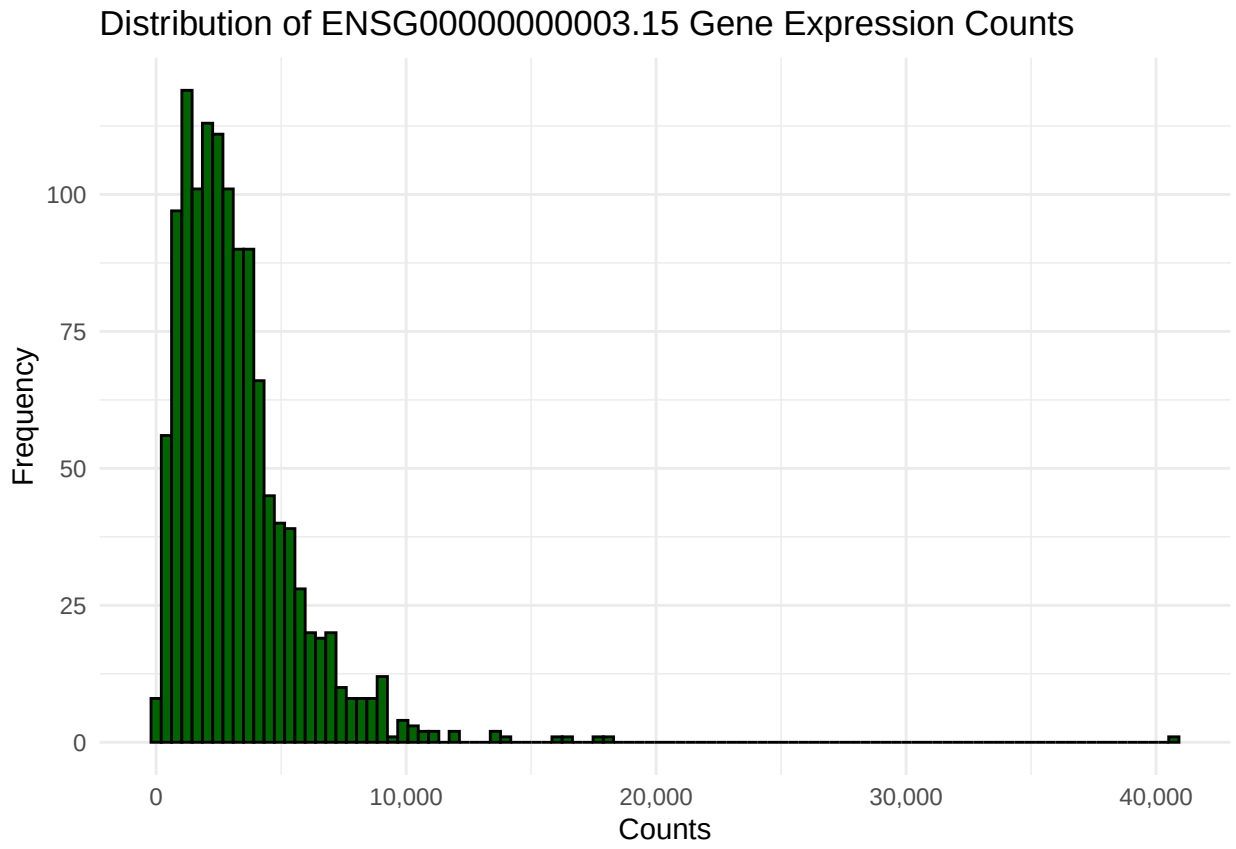
```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

```
library(scales) # Using this for the comma
```

```
## Warning: package 'scales' was built under R version 4.3.3
```

```
# Making counts into DF for ggplot
df_hist <- data.frame(counts = gene1_counts)

# Histogram
ggplot(df_hist, aes(x = counts)) +
  geom_histogram(bins = 100, fill = "darkgreen", color = "black") +
  labs(
    title = "Distribution of ENSG00000000003.15 Gene Expression Counts",
    x = "Counts",
    y = "Frequency"
  ) +
  theme_minimal() +
  scale_x_continuous(labels = comma)
```

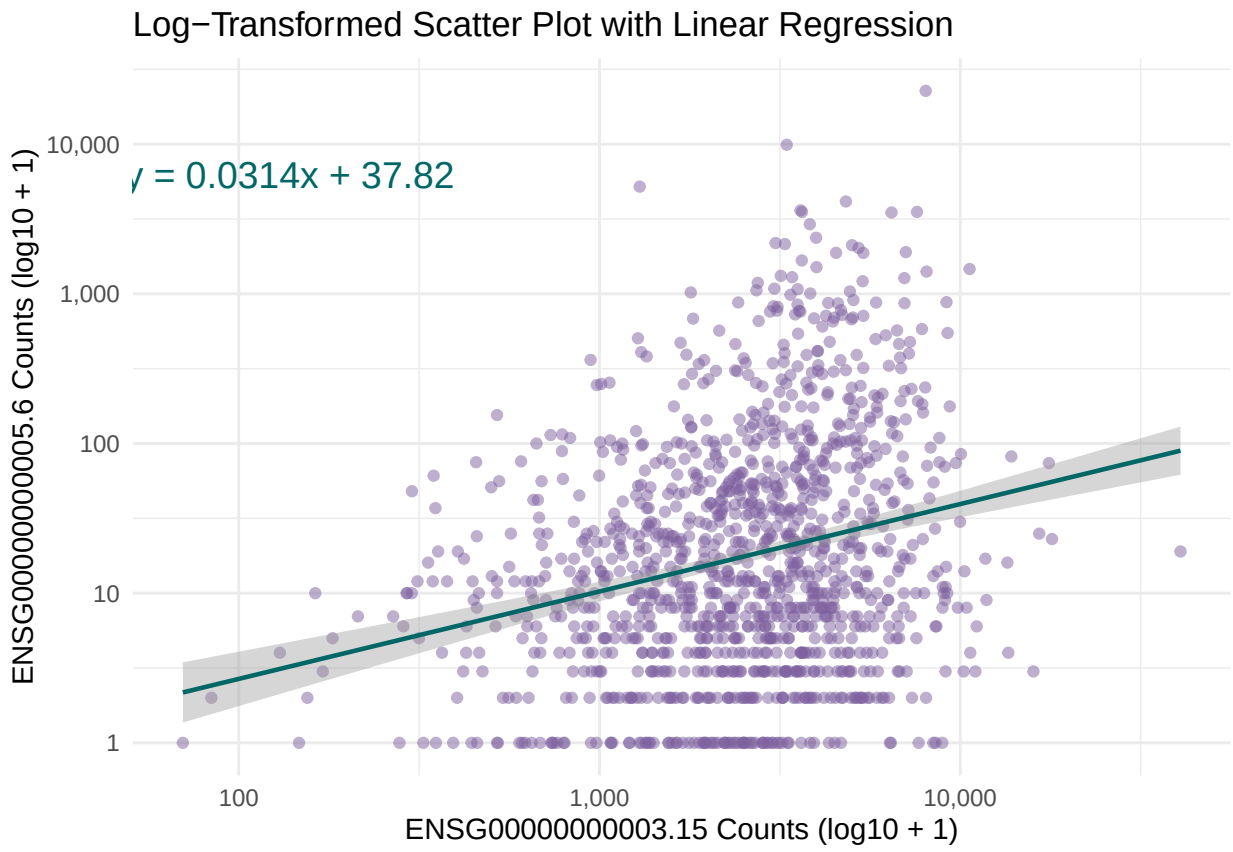


b. Create a Scatter Plot: Select a second gene from the dataset. Create a scatter plot using ggplot2 to compare the count data of the two selected genes.

```
library(ggplot2)
library(scales)

# Data with pseudocounts to avoid log(0)
df_scatter <- data.frame(Gene1 = gene1_counts + 1, Gene2 = gene2_counts + 1)

# Fitting linear model
model <- lm(Gene2 ~ Gene1, data = df_scatter)

# Extra coeff for eq label
coefficients <- coef(model)
intercept <- coefficients[1]
slope <- coefficients[2]

# Create eq label text
eq_label <- paste0("y = ",
                   round(slope, 4), "x + ",
                   round(intercept, 2))

# Scatter plot
ggplot(df_scatter, aes(x = Gene1, y = Gene2)) +
  geom_point(alpha = 0.5, color = "#7E619F") +
```

```r
  geom_smooth(method = "lm", se = TRUE, color = "#006666", linewidth = 0.8) +
  scale_x_log10(labels = scales::comma) +
  scale_y_log10(labels = scales::comma) +
  labs(
    title = "Log-Transformed Scatter Plot with Linear Regression",
    x = "ENSG00000000003.15 Counts (log10 + 1)",
    y = "ENSG00000000005.6 Counts (log10 + 1)"
  ) +
  annotate(
    "text",
    x = min(df_scatter$Gene1, na.rm = TRUE) * 7, # eq position horizontal
    y = max(df_scatter$Gene2, na.rm = TRUE) / 7, # eq position vertical
    label = eq_label,
    hjust = 1.1, vjust = -1.1,
    size = 5,
    color = "#006666"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

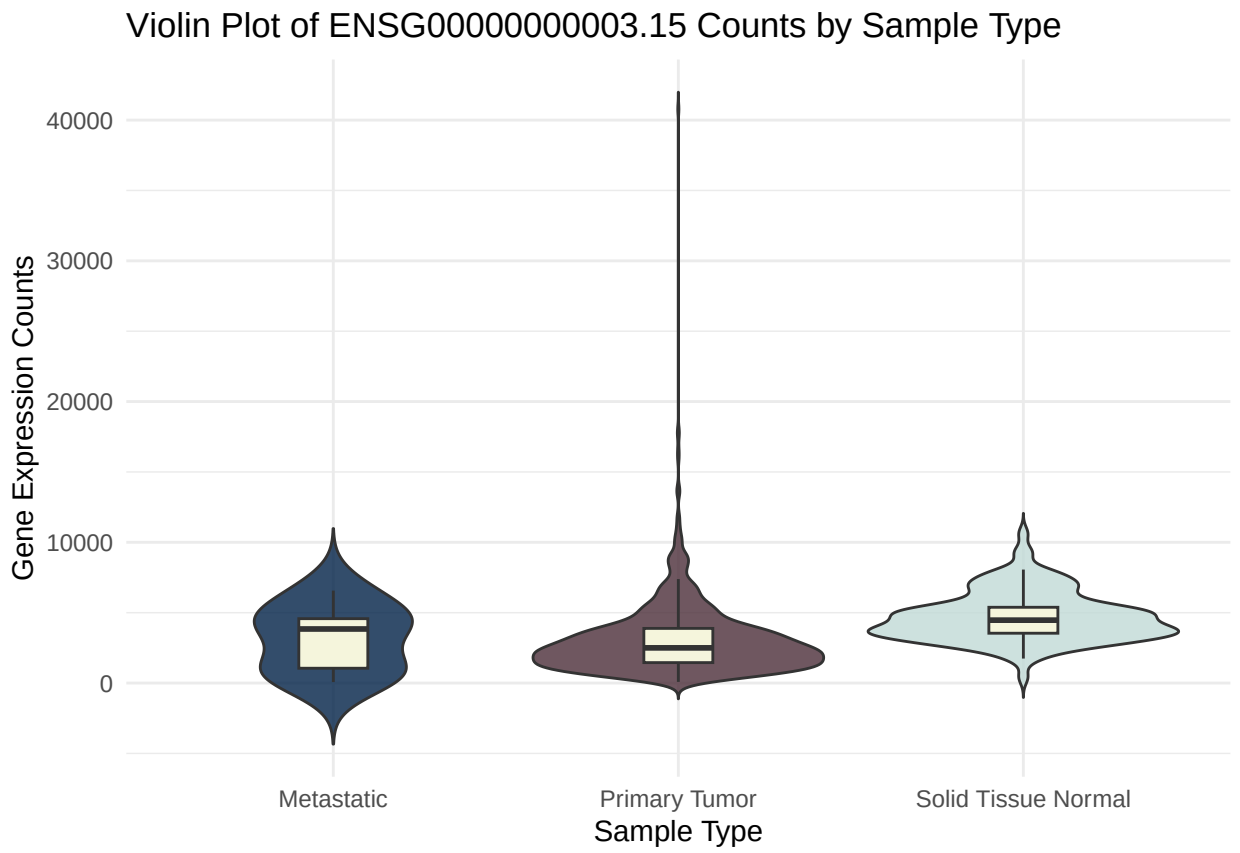


c. Create a Violin Plot: Select one covariate from your metadata. Using the count data from the first gene and the selected covariate, generate a violin plot that illustrates the distribution of count data based on the covariate. For example, if you choose "primary_diagnosis", your plot should display a violin plot for each level in "primary_diagnosis".

```
# ^ trying to adjust the output fig height for better view

# Setting the covariate for reusable code, trying the example
#covariate <- metadata$primary_diagnosis

# Combine into df
#df_violin <- data.frame(
  #Counts = gene1_counts,
  #Covariate = covariate
#)

#ggplot(df_violin, aes(x = Covariate, y = Counts, fill = Covariate)) +
  #geom_violin(alpha = 0.7, trim = FALSE) +
  #geom_boxplot(width = 0.1, fill = "white", outlier.shape = NA) + # overlay boxplot to see the values
  #labs(
   # title = "Violin Plot of ENSG00000000003.15 Counts by Primary Diagnosis",
   # x = "Primary Diagnosis",
   # y = "Gene Expression Counts"
 # ) +
 # theme_minimal() +
 # theme(
  #  axis.text.x = element_text(angle = 45, hjust = 1, size = 8),  # tilt labs for fit
  #  legend.position = "none"                                      # remove legend
#  )
```

```
library(ggplot2)

# Making my own color palette
sample_colors <- c(
  "Metastatic" = "#002147",
  "Primary Tumor" = "#4B2E39",
  "Solid Tissue Normal" = "#C1DAD6"
)

# DF for the
covariate <- metadata$sample_type

df_violin <- data.frame(
  Counts = gene1_counts,
  Covariate = covariate
)

# Violin plot
ggplot(df_violin, aes(x = Covariate, y = Counts, fill = Covariate)) +
  geom_violin(alpha = 0.8, trim = FALSE) +
  geom_boxplot(width = 0.2, fill = "#F5F5DC", outlier.shape = NA) +
  scale_fill_manual(values = sample_colors) +
  labs(
    title = "Violin Plot of ENSG00000000003.15 Counts by Sample Type",
    x = "Sample Type",
    y = "Gene Expression Counts"
  ) +
  theme_minimal() +
```

```
  theme(legend.position = "none")
```



4. Heatmap Analysis: Select 10 genes: Choose a set of 10 different genes from the count matrix for your heatmap.

```
library(ComplexHeatmap)
```

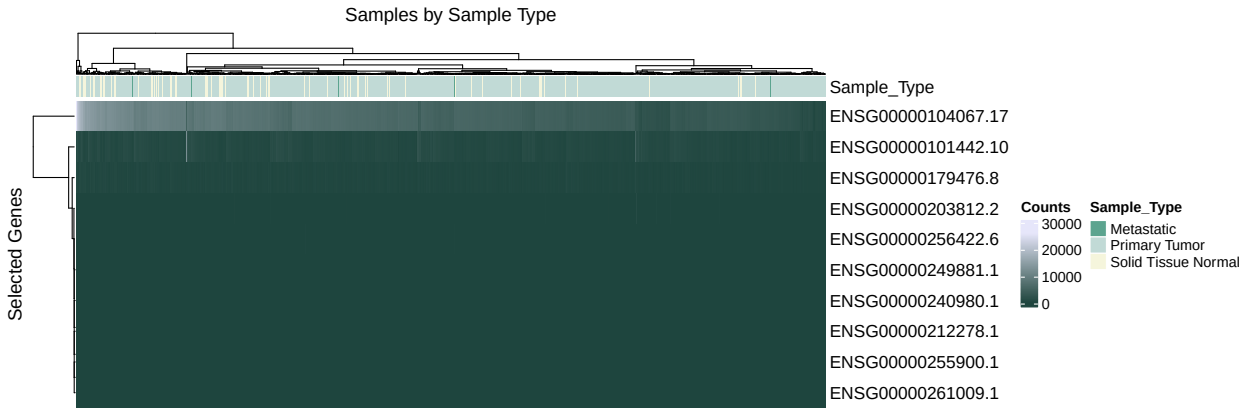
```
## Loading required package: grid

## ========================================
## ComplexHeatmap version 2.18.0
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##     genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
```

```
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## ========================================
```

```
suppressPackageStartupMessages(library(circlize)) # Had some messages I didnt want to print

set.seed(101)
selected_genes <- sample(rownames(count_data), 10)
heatmap_data <- count_data[selected_genes, ]
```

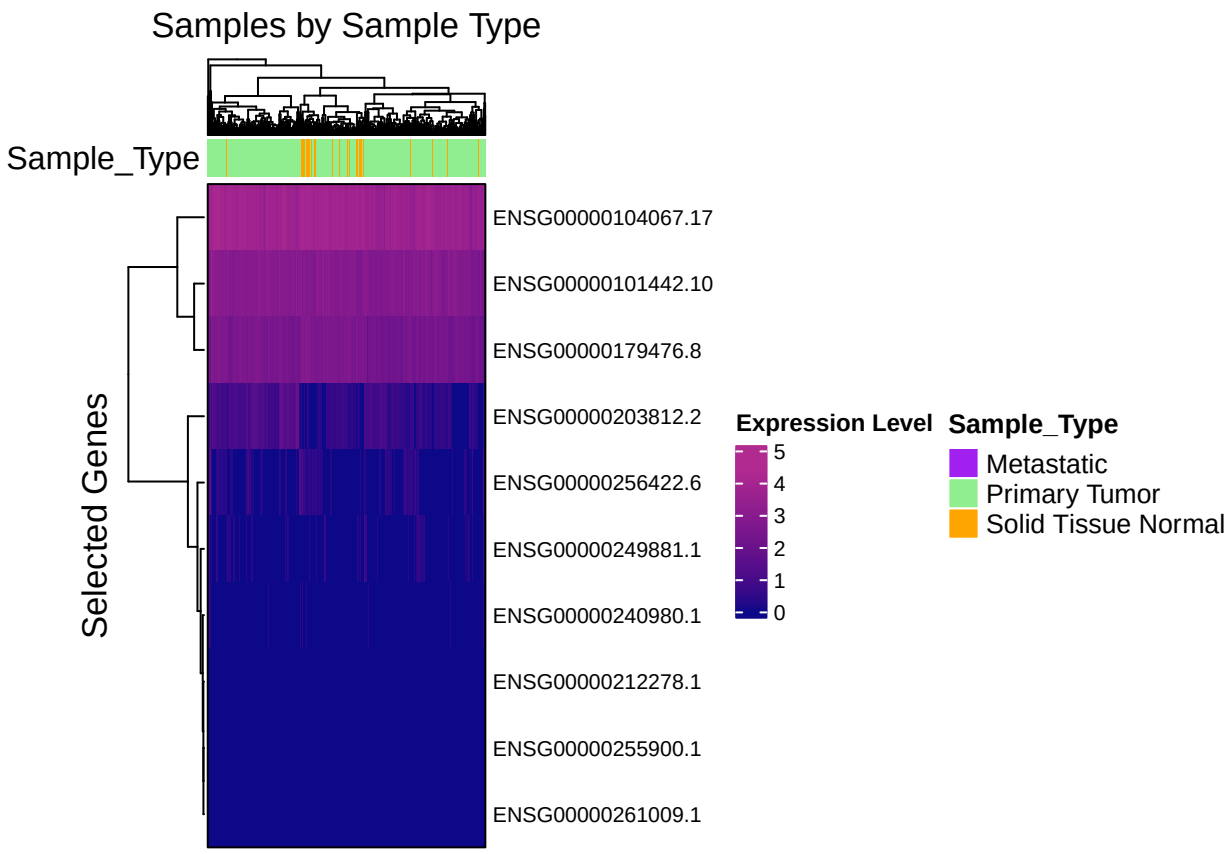
Generate a Heatmap: Use the ComplexHeatmap package in R to create a heatmap of the count data for the selected genes. Add an Annotation Bar: Include an annotation bar reflecting your chosen covariate for further context and interpretation of the data.

```
# Annotation with sample_type
column_ha <- HeatmapAnnotation(
  Sample_Type = metadata$sample_type,
col = list(
  Sample_Type = c(
    "Primary Tumor" = "#C1DAD6",
    "Solid Tissue Normal" = "#F5F5DC",
    "Metastatic" = "#5ca48f"))
)

# Create heatmap
Heatmap(
  as.matrix(heatmap_data),
  name = "Expression",
  top_annotation = column_ha,
  show_row_names = TRUE,
  show_column_names = FALSE,
  cluster_rows = TRUE,
  cluster_columns = TRUE,
  row_title = "Selected Genes",
  column_title = "Samples by Sample Type",
  col = colorRamp2(
    c(min(heatmap_data), median(as.matrix(heatmap_data)), max(heatmap_data)),
    c("#1e453e", "#95b9c7", "lavender")
  ),
  heatmap_legend_param = list(title = "Counts")
)
```




```r
library(ComplexHeatmap)
suppressPackageStartupMessages(library(circlize))

# Log-transform expression counts to reduce skew
heatmap_data_log <- log10(heatmap_data + 1)

# Define annotation colors for sample types
sample_type_colors <- c(
  "Primary Tumor" = "lightgreen",
  "Solid Tissue Normal" = "orange",
  "Metastatic" = "purple"            # purple tone for contrast
)

# Create annotation bar
column_ha <- HeatmapAnnotation(
  Sample_Type = metadata$sample_type,
  col = list(Sample_Type = sample_type_colors),
  annotation_name_side = "left"
)

# Define a strong, diverging color palette for expression levels
color_scale <- colorRamp2(
  c(min(heatmap_data_log), median(as.matrix(heatmap_data_log)), max(heatmap_data_log)),
  c("#0D0887", "#F0F921", "#B12A90")
)

# Build heatmap
Heatmap(
  as.matrix(heatmap_data_log),
  name = "Log10(Expression + 1)",
  top_annotation = column_ha,
  show_row_names = TRUE,
  row_names_gp = gpar(fontsize = 8),
  show_column_names = FALSE,
  cluster_rows = TRUE,
  cluster_columns = TRUE,
  border = TRUE,
  row_title = "Selected Genes",
  column_title = "Samples by Sample Type",
  col = color_scale,
```

```
  heatmap_legend_param = list(
    title = "Expression Level",
    title_gp = gpar(fontsize = 9, fontface = "bold"),
    labels_gp = gpar(fontsize = 8)
  )
)
```




```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.3.3
```

```
## corrplot 0.95 loaded
```

```
# log transform and keep numeric
corr_data <- log10(heatmap_data + 1)

# remove genes with zero variance
corr_data <- corr_data[apply(corr_data, 1, sd) != 0, ]

# recompute corr
corr_matrix <- cor(t(corr_data), use = "pairwise.complete.obs", method = "pearson")

# draw corr matrix
library(corrplot)
```

```r
corrplot(
  corr_matrix,
  method = "color",
  type = "upper",
  order = "hclust",
  addCoef.col = "black",
  tl.col = "black",
  tl.srt = 45,
  col = colorRampPalette(c("#67001f", "#f7f7f7", "#053061"))(200),
  title = "Correlation Matrix of Selected Genes",
  mar = c(0, 0, 2, 0)
)
```